완료 보고서

AXI_Peripheral

2026.06.22~2026.06.30

한승훈

1. 개요 (Overview) **4**
1.1 목적 및 목표 4
1.2 설계 사양 요약 4
2. 프로젝트 관리 (Project Management) **5**
2.1 개발 환경 5
2.2 담당 역할 5
2.3 개발 일정 6
3. 아키텍처 설계 (Architecture) **6**
3.1 시스템 구조 (System Architecture) 7
3.2 실무 설계 이론 (RTL/SoC 관점) 9
3.2.1 AXI4-Lite Interface 9
3.2.2 Memory Map과 Register Access 10
3.2.3 H/W-S/W Co-design 구조 11
3.2.4 Peripheral Integration 12
4. 상세 설계 (Detailed Design) **12**
4.1 GPIO, Timer, Custom UART 12
4.2 I2C LCD Peripheral 13
4.3 SPI AXI-Lite Peripheral 14
4.4 SPI Register Map 15
4.5 SPI Software Access Sequence 16
5. 시뮬레이션 및 검증 (Simulation & Verification) **17**
5.1 검증 목표 및 전략 17
5.2 SPI UVM Architecture 17
5.3 SPI UVM Verification Scenario 18
5.4 검증 결과 19
5.5 Board Validation Overview 20
5.6 UART & SPI Board-to-Board Validation 20
5.7 I2C LCD Board Validation 21
6. 결과 분석 및 트러블슈팅 (Analysis & Trouble shooting) **22**
6.1 종합 결과 분석 22
6.2 문제 원인 분석 및 해결 22
6.3 개선 및 고도화 방향 23
7. 결론 **23**
7.1 결론 및 고찰 23
7.2 향후 개선 및 고도화 방향 24

1. 개요 (Overview)

1.1 목적 및 목표

본 프로젝트는 Basys3 FPGA 내부에 MicroBlaze 기반의 Software-Controlled SoC를 구성하고, AXI4-Lite를 이용해 여러 주변장치를 Memory-Mapped 방식으로 제어하는 구조를 설계·구현·검증하는 것을 목표로 하였다.

MicroBlaze에서 실행되는 Vitis C Application은 각 Peripheral의 Base Address와 Register Offset을 이용해 제어 및 상태 레지스터를 읽고 쓴다. 이를 통해 Software의 명령이 실제 하드웨어의 address decode, VALID/READY handshake, start pulse, data 및 status 신호로 변환되는 전 과정을 확인하였다.

통합 대상은 GPIO, Timer, Custom UART, I2C LCD, SPI이며, SPI Peripheral은 SystemVerilog/UVM 환경에서 별도로 검증하였다. 이후 Basys3 두 대를 연결하여 UART와 SPI 통신을 수행하고 LCD·LED·FND 표시 결과까지 확인함으로써 시뮬레이션 결과가 실제 보드에서도 동일하게 동작하는지 검증하였다.

1.2 설계 사양 요약

- Processor: MicroBlaze 32-bit Soft Processor
- Bus: AMBA AXI4-Lite, Memory-Mapped Single Transfer
- Board / Clock: Basys3 2대, 100 MHz System Clock
- Peripheral: GPIO, I2C LCD, Custom UART, SPI, Timer
- Verification: SystemVerilog/UVM, Scoreboard, Functional/Code Coverage, Board Validation

2. 프로젝트 관리 (Project Management)

2.1 개발 환경

구분	내용
RTL / Verification	Verilog, SystemVerilog, UVM
FPGA Design	Xilinx Vivado, IP Integrator, Vivado Simulator
Firmware	Vitis Embedded C, MicroBlaze
Target	Basys3 (Artix-7), 100 MHz
Interface	AXI4-Lite, GPIO, I2C, UART, SPI

2.2 담당 역할

- MicroBlaze 기반 AXI4-Lite Multi-Peripheral SoC 구성
- GPIO, Timer, Custom UART, I2C LCD, SPI Peripheral 통합
- Peripheral별 Memory Map 및 Register Map 정의
- Vitis C Application과 Driver/HAL 계층 작성
- SPI UVM Testbench, Monitor, Scoreboard, Coverage 구성
- Basys3 2대를 이용한 UART/SPI Board-to-Board 검증
- I2C LCD, LED, FND를 이용한 결과 표시 및 디버깅
- 발표 자료와 완료보고서 작성

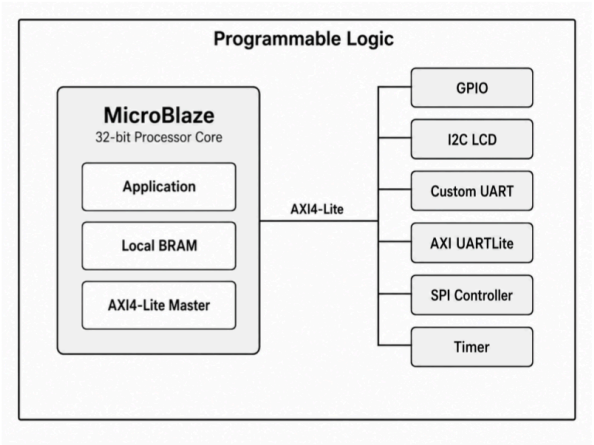
2.3 개발 일정

기간	단계	주요 수행 내용
06.22	기획 및 환경 구성	목표 정의, MicroBlaze·AXI4-Lite 구조 학습, 개발 환경 구성
06.23	SoC 아키텍처 설계	MicroBlaze, AXI Interconnect, Peripheral 구성 및 Address Map 정의
06.24	AXI4-Lite 설계	Slave Register Block, read/write handshake 및 WSTRB 처리
06.25	Peripheral 통합	GPIO, Timer, Custom UART와 C Driver 연결
06.26	I2C LCD 통합	I2C LCD Register Block, 제어 시퀀스 및 표시 검증
06.27	SPI AXI-Lite 설계	SPI Register Map, START pulse, BUSY/DONE 제어
06.28	SPI UVM 검증	AXI Agent, SPI Monitor, Scoreboard, Coverage 구성
06.29	Board Validation	UART/SPI 보드 간 통신 및 LCD·LED·FND 출력 확인
06.30	분석 및 문서화	결과 분석, 트러블슈팅 정리, 발표 및 완료보고서 작성

3. 아키텍처 설계 (Architecture)

전체 시스템은 Programmable Logic 내부에 MicroBlaze, Local BRAM, AXI4-Lite Master 및 AXI Interconnect를 배치하고, 여러 Custom Peripheral을 AXI4-Lite Slave로 연결한 구조이다.

Overview



프로젝트 목표

- MicroBlaze 기반 Software-Controlled SoC 구조 학습
- AXI4-Lite를 통한 Memory-Mapped Peripheral 제어 이해
- Vitis C Application과 Peripheral Register 연동
- I2C LCD / SPI Custom Peripheral 통합 및 Board 검증

그림 1. MicroBlaze 기반 Software-Controlled SoC 구조

3.1 시스템 구조 (System Architecture)

- MicroBlaze: Local BRAM의 C Application을 실행하고 AXI4-Lite Master transaction을 생성한다.
- AXI Interconnect: Master 주소를 해석하여 선택된 Peripheral Slave로 transaction을 전달한다.
- Register Block: AXI address decode, write strobe, readback 및 response를 처리한다.
- Core Logic: UART baud timing, SPI clock, Timer counter, I2C command sequence 등 장치별 기능을 수행한다.
- Board I/O: Switch와 Button을 입력으로 사용하고 LCD, LED, FND 및 UART/SPI 신호로 결과를 확인한다.

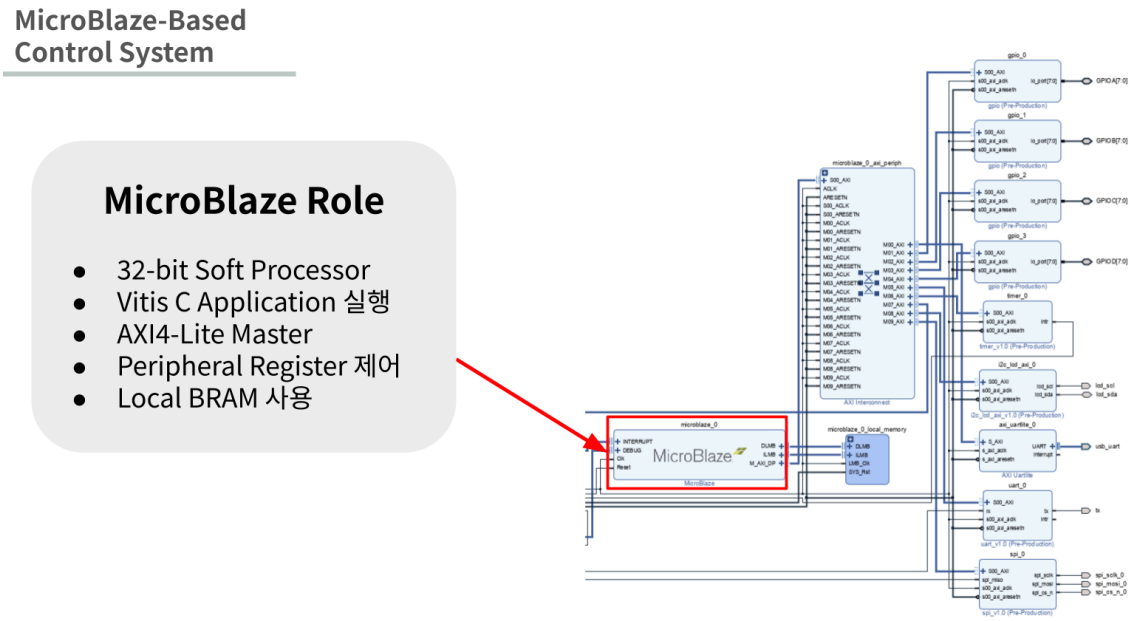


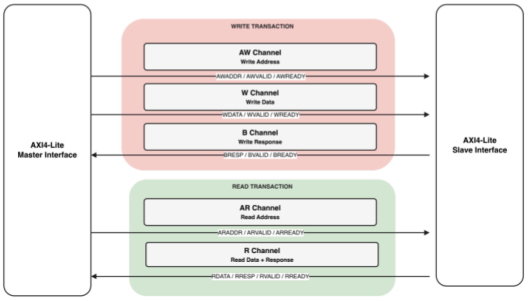
그림 2. MicroBlaze Block Design 및 역할

Software는 Base Address와 Offset을 조합해 레지스터를 접근하고, 하드웨어 Register Block은 이를 start, data, status 신호로 변환한다. 따라서 CPU 관점의 단순한 read/write가 실제 Peripheral의 timing 제어로 이어진다.

3.2 실무 설계 이론 (RTL/SoC 관점)

3.2.1 AXI4-Lite Interface

AXI4-Lite Interface



Item	AXI4-Lite	APB
Control	VALID / READY	PSEL / PENABLE
Structure	5 channels	Setup / Access
Use	AXI SoC peripheral	Simple peripheral
Burst	No	No

AXI4-Lite 특징

- Memory-mapped register access
- 5 independent channels: AW / W / B / AR / R
- Single read/write transaction
- No burst transfer
- Transfer when VALID && READY

그림 3. AXI4-Lite 5채널 구조

AXI4-Lite는 Memory-Mapped Register Access에 적합한 경량 AMBA 인터페이스이다. Burst를 지원하지 않으며 각 read/write는 단일 transfer로 처리된다. Write는 AW 주소, W 데이터, B 응답 채널로 구성되고 Read는 AR 주소와 R 데이터·응답 채널로 구성된다.

각 채널에서 VALID와 READY가 같은 사이클에 1이 되는 시점을 실제 transfer 완료로 판단하였다. Address와 Data 채널은 독립적으로 handshake될 수 있으므로 Register Block에서는 각 handshake의 완료 여부를 분리해서 관리하도록 설계하였다.

3.2.2 Memory Map과 Register Access

Memory-Mapped Address Map

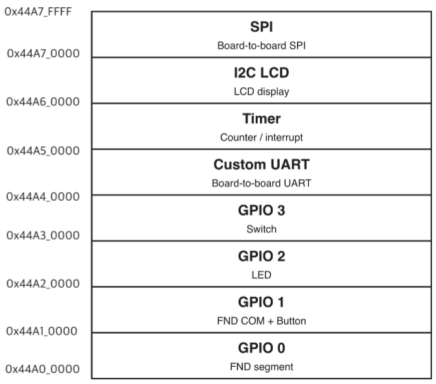


그림 4. AXI4-Lite Peripheral Address Map

- 0x44A0_0000: GPIO 0 — FND Segment
- 0x44A1_0000: GPIO 1 — FND COM / Button
- 0x44A2_0000: GPIO 2 — LED
- 0x44A3_0000: GPIO 3 — Switch
- 0x44A4_0000: Custom UART
- 0x44A5_0000: Timer
- 0x44A6_0000: I2C LCD
- 0x44A7_0000: SPI

각 Peripheral은 독립된 64 KB 주소 공간에 배치하였다. 상위 주소로 Peripheral을 선택하고 하위 Offset으로 Control, Data, Status Register를 구분하여 하드웨어와 Firmware가 동일한 인터페이스 규격을 공유하도록 하였다.

3.2.3 H/W-S/W Co-design 구조

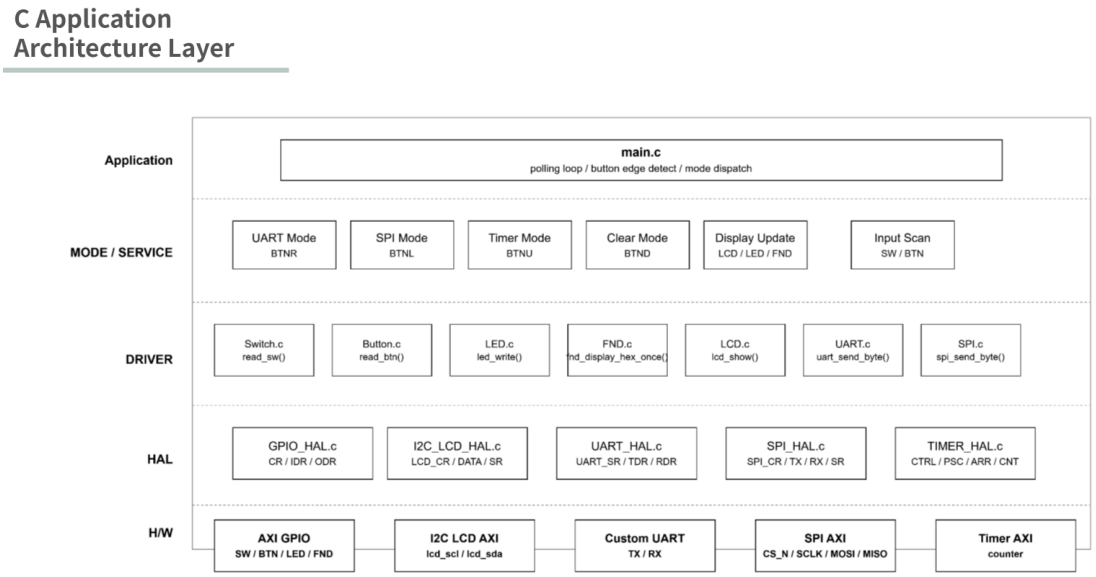


그림 5. Application-Service-Driver-HAL-Hardware 계층

C Application은 UI와 Mode 선택을 담당하고, Service 계층은 UART 전송, SPI 전송, LCD 표시 등 기능 단위의 순서를 조합한다. Driver/HAL은 Base Address, Register Offset 및 Bit Field를 캡슐화하여 상위 Application이 하드웨어 주소를 직접 반복해서 다루지 않도록 구성하였다.

이러한 계층화를 통해 Address Map이나 Register Map이 변경되더라도 상위 로직의 수정 범위를 줄일 수 있었고, Software 문제와 RTL 문제를 분리하여 디버깅할 수 있었다.

3.2.4 Peripheral Integration

AXI4-Lite Peripheral Integration

Peripheral	Register Block	Core Logic	Role
GPIO	CR / IDR / ODR	Direction Control	SW / BTN / LED / FND
I2C LCD	**CR / DATA / SR**	**I2C LCD Core**	**LCD Mode & Data Display**
Custom UART	SR / TDR / RDR / CR	UART TX/RX	Board-to-Board UART
SPI	**CR / TX / RX / SR**	**SPI Master**	**Board-to-Board SPI**
Timer	CTRL / PSC / ARR / CNT	Counter	Timer Counter

그림 6. Peripheral별 Register Block과 Core Logic

각 Peripheral은 AXI4-Lite Slave Register Block과 기능 Core Logic으로 분리하였다. Register Block은 bus protocol을 담당하고 Core Logic은 장치별 timing을 담당한다. 디버깅 시에는 AXI handshake와 register 값을 먼저 확인한 뒤, Core의 start/status 신호와 외부 I/O 파형을 순서대로 확인하였다.

4. 상세 설계 (Detailed Design)

4.1 GPIO, Timer, Custom UART

GPIO는 FND Segment, FND COM/Button, LED, Switch를 각각 독립 주소에 배치하였다. Firmware는 Switch와 Button을 읽어 동작 모드를 선택하고 수신 데이터 또는 상태를 LED와 FND에 표시한다.

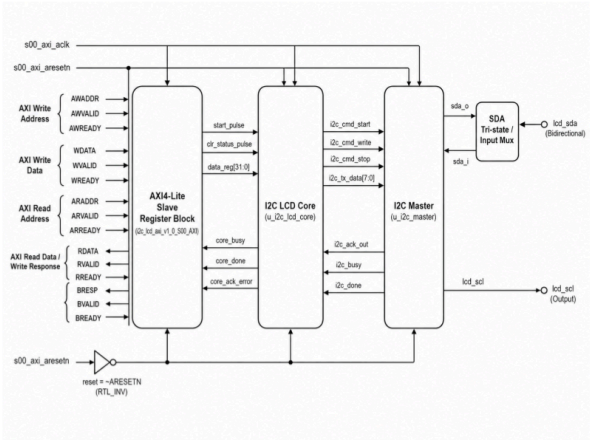
Timer는 CTRL, Prescaler(PSC), Auto-Reload(ARR), Counter(CNT) 형태의 레지스터로 제어하였다. 본 Application에서는 Counter와 Status를 반복해서 확인하는 Polling 방식을 사용했으며 Interrupt 기반 제어는 향후 개선 항목으로 남겼다.

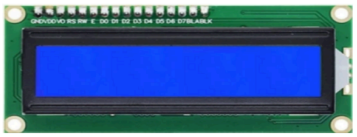
- CR: Control 및 Interrupt Enable — 초기화와 동작 설정
- SR: TX/RX 상태 — Ready/Done 상태 확인
- TDR: Transmit Data — 전송할 8-bit 데이터 기록
- RDR: Receive Data — 수신된 8-bit 데이터 확인

Custom UART는 송신측과 수신측의 Baud Rate를 동일하게 설정하고, 전송 중 중복 write를 방지하기 위해 Busy/Ready 상태를 확인한 뒤 TDR에 데이터를 기록하도록 구성하였다.

4.2 I2C LCD Peripheral

I2C LCD Peripheral Structure





I2C LCD Control Flow

1. Write mode/data + START
2. LCD Core generates command sequence
3. I2C Master drives SCL/SDA to LCD

그림 7. I2C LCD Peripheral 구조와 제어 흐름

I2C LCD Peripheral은 AXI4-Lite Register Block, I2C LCD Control 및 I2C Master로 구성된다. Firmware가 표시할 Mode와 Data를 기록하고 START를 요청하면 Control Block이 LCD Command Sequence를 생성하고 I2C Master가 SCL과 SDA를 구동한다.

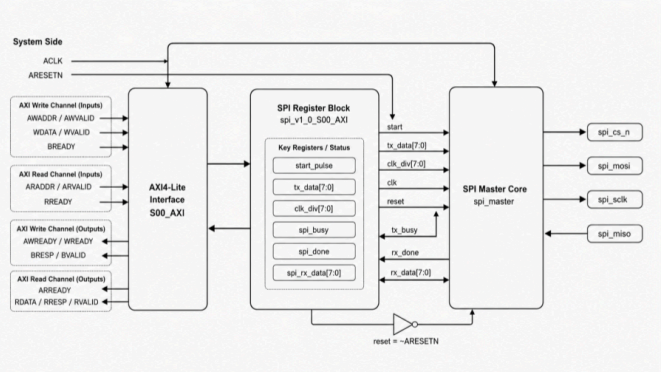
- Software가 표시할 Mode와 Data를 DATA Register에 기록한다.

- CR의 START Bit를 Write하여 One-Cycle Start Event를 발생시킨다.
- LCD Control이 Command, Data 및 Enable Sequence를 생성한다.
- I2C Master가 SCL/SDA를 통해 LCD로 명령과 데이터를 전달한다.
- Software가 Busy/Done 상태를 확인하고 다음 Command를 수행한다.

LCD 초기화와 Command 사이 Delay가 부족하면 ACK가 정상이어도 화면이 갱신되지 않을 수 있으므로 Bus Handshake뿐 아니라 LCD 자체 Timing까지 함께 확인하였다.

4.3 SPI AXI-Lite Peripheral

SPI AXI-Lite Peripheral Overview



- AXI4-Lite register block converts software writes into SPI control signals
- start_pulse, tx_data, clk_div are generated by register access
- SPI master drives CS_n, SCLK, MOSI and samples MISO

그림 8. SPI AXI-Lite Register Block과 SPI Master

SPI Peripheral은 AXI4-Lite Register Write를 SPI 제어 신호로 변환한다. TX Data와 Clock Divider는 Register에 저장되고, CR의 START Write Event는 One-Cycle Start Pulse로 변환되어 SPI Master를 기동한다.

START Bit를 Level 신호로 직접 연결하면 AXI Write 이후 여러 Cycle 동안 SPI Master가 반복 기동될 수 있다. 따라서 CR Register의 Write Event에서 Start Pulse를 생성하고, 전송 중에는 BUSY를 확인하여 추가 START 요청을 차단하였다. DONE은 Software가 확인할 수 있도록 유지한 뒤 CLR_STATUS Write로 지우도록 구성하였다.

RX_DATA는 SPI 전송 완료 시점에만 갱신하여 Software가 전송 중간값을 읽는 문제를 방지하였다.

4.4 SPI Register Map

SPI Register Map

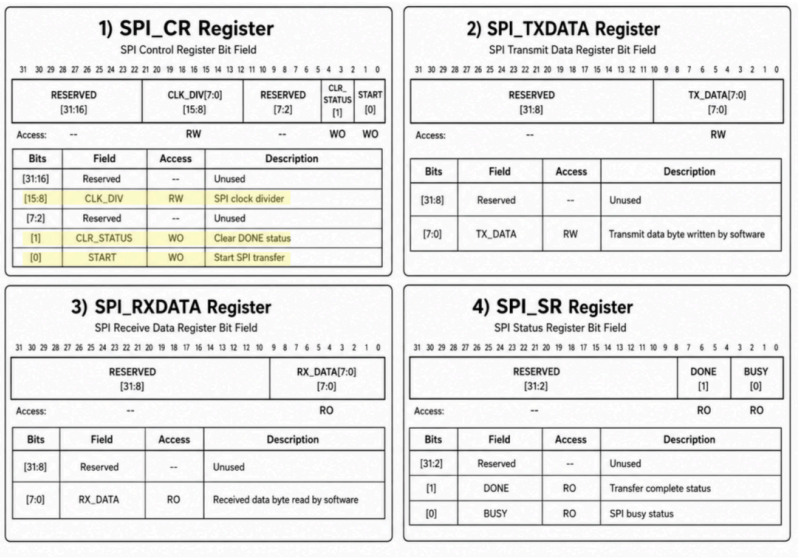


그림 9. SPI Control/TX/RX/Status Register Bit Field

- 0x00 SPI_CR: [0] START, [1] CLR_STATUS — Write Only Command
- 0x04 SPI_TXDATA: [7:0] TX Data — Write Only
- 0x08 SPI_RXDATA: [7:0] RX Data — Read Only
- 0x0C SPI_SR: [0] DONE, [1] BUSY — Read Only
- 0x10 SPI_CLKDIV: Clock Divider 값 — Read/Write

Control과 Status를 분리하고 Write-Only Command Bit와 Read-Only Status Bit의 접근 속성을 명확히 정의하였다. Reserved Bit는 Read 시 0을 반환하고 Write 영향이 없도록 처리하였다.

4.5 SPI Software Access Sequence

SPI Access Sequence

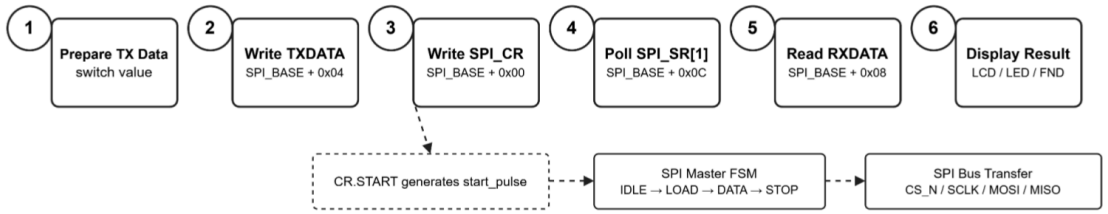


그림 10. SPI Register Access와 Bus Transfer Sequence

- 전송 데이터를 SPI_TXDATA Register에 기록한다.
- 필요한 Clock Divider 값을 SPI_CLKDIV Register에 설정한다.
- SPI_CR의 START Bit를 Write하여 전송을 시작한다.
- SPI_SR의 BUSY와 DONE을 Polling하며 완료를 기다린다.
- DONE 확인 후 SPI_RXDATA에서 수신 데이터를 읽는다.
- CLR_STATUS를 Write하여 DONE 상태를 정리한다.

Firmware는 TXDATA Write → START Write → DONE Polling → RXDATA Read 순서를 유지하며, 비정상 BUSY 상태에서 무한 대기하지 않도록 Timeout을 함께 적용할 필요가 있다.

5. 시뮬레이션 및 검증 (Simulation & Verification)

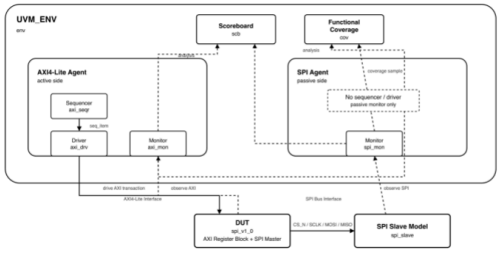
5.1 검증 목표 및 전략

검증은 Register-Level, Protocol-Level, System-Level의 세 계층으로 나누어 수행하였다. AXI Write/Read가 올바른 Register 동작으로 변환되는지, SPI Bus가 설정된 Sequence로 동작하는지, 실제 보드에서 End-to-End Data가 일치하는지를 단계적으로 확인하였다.

- Register-Level: AXI4-Lite AW/W/B/AR/R Handshake와 Register Readback 확인
- Protocol-Level: CS_n, SCLK, MOSI, MISO와 START·BUSY·DONE 상태 순서 확인
- Data Integrity: TX_DATA와 MOSI, MISO와 RX_DATA의 값 비교
- Corner Case: 0x00, 0xFF, 0xAA, 0x55, Random Byte 및 Clock Divider 경계값
- System-Level: Vitis Application부터 UART/SPI Board Link 및 LCD·LED·FND 출력까지 확인

5.2 SPI UVM Architecture

SPI UVM Architecture



Component	Main Role	Output
axi_lite_sequence	SPI 전송 시나리오 생성	tx_data, slave_tx_data, clk_div
axi_lite_driver	AXI4-Lite register write/read 구동	DUT AXI Interface
axi_lite_monitor	AXI write/read transaction 관찰	Scoreboard / Coverage
spi_monitor	CS_N, SCLK, MOSI, MISO 관찰	Scoreboard / Coverage
scoreboard	AXI 데이터와 SPI bus 데이터 비교	Pass / Fail
coverage	주요 기능 coverpoint 수집	Coverage Report

그림 11. AXI-Lite Agent와 SPI Monitor 기반 UVM 환경

- Sequence/Sequencer: AXI Register Access Transaction 생성

- AXI Driver: AW/W/B/AR/R Channel을 구동하여 DUT Register 접근
- AXI Monitor: Software가 의도한 Register Address와 Data 수집
- SPI Monitor: CS_n, SCLK, MOSI, MISO에서 실제 Serial Data 복원
- Scoreboard: AXI Register Data와 SPI Bus Data의 Expected/Actual 비교
- Coverage: 기능 항목, Data Pattern, Status 및 SPI Bus Event 집계

AXI Monitor와 SPI Monitor를 분리하여 Software가 Register에 의도한 값과 실제 Serial Bus에서 관찰된 값을 독립적으로 수집하고 Scoreboard에서 비교하였다.

5.3 SPI UVM Verification Scenario

SPI UVM Verification

Step	Action	Register / Signal	Expected Behavior
1	Create transfer item	tx_data, slave_tx_data, clk_div	전송 시나리오 생성
2	Write TX data	SPI_TXDATA 0x04	MOSI로 전송할 데이터 저장
3	Set slave response	slave_tx_data	MISO 응답 데이터 준비
4	Start transfer	SPI_CR 0x00	START pulse 발생, SPI 전송 시작
5	Poll done	SPI_SR 0x0C	DONE == 1까지 대기
6	Read RX data	SPI_RXDATA 0x08	MISO에서 수신한 데이터 확인
7	Check result	Scoreboard	TX/MOSI, RX/MISO 비교

그림 12. SPI UVM Directed Verification Scenario

- Reset 후 Register와 Status가 초기값으로 복귀하는지 확인한다.
- SPI_TXDATA와 SPI_CLKDIV에 시험 값을 기록한다.
- SPI_CR의 START Write를 수행하고 BUSY Assert를 확인한다.
- SPI Monitor가 MOSI/MISO Byte를 수집하고 전송 종료를 검출한다.
- DONE Assert와 RX_DATA Update 시점을 확인한다.
- Scoreboard에서 TX/MOSI 및 RX/MISO Data를 비교한다.
- CLR_STATUS 이후 DONE이 정상적으로 Clear되는지 확인한다.

추가로 BUSY 중 START 재요청, 연속 Transfer, Reserved Bit Write, Invalid Offset Access, Reset 도중 및 Reset 해제 직후 Transaction을 수행하여 상태와 Monitor Queue가 정상적으로 초기화되는지 확인하였다.

5.4 검증 결과

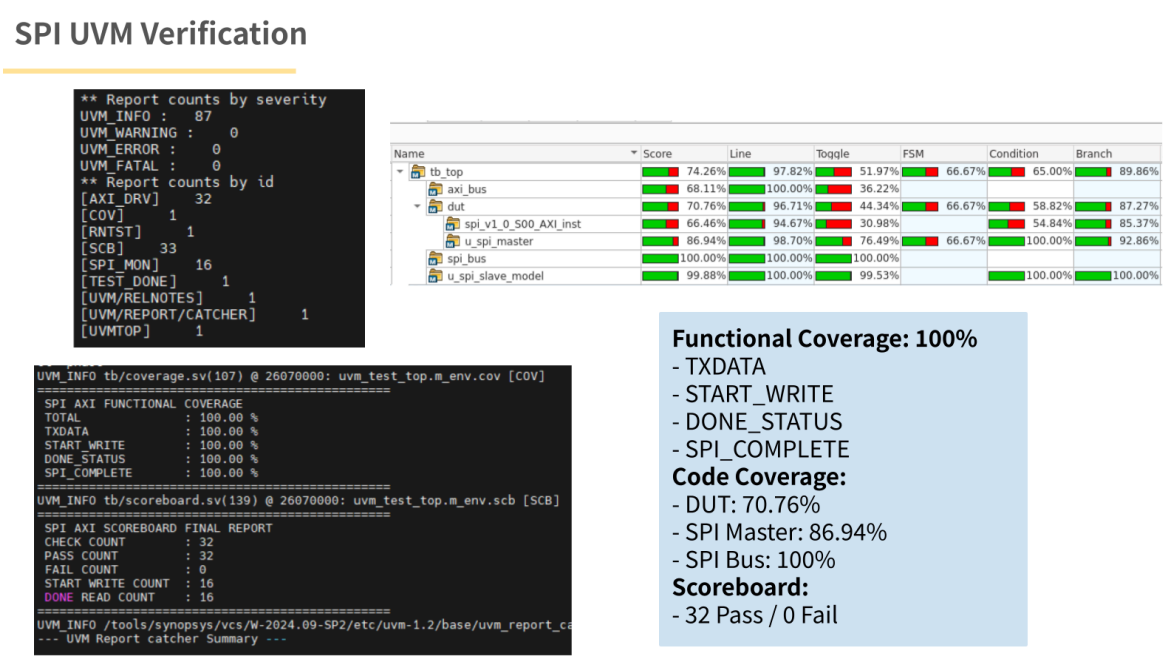


그림 13. UVM Scoreboard 및 Coverage 결과

검증 항목	결과
Functional Coverage	100% — TXDATA, START, WRITE_DONE, STATUS, SPI_COMPLETE
SPI Bus Coverage	100% — CS_n, SCLK, MOSI, MISO Sequence
Scoreboard	32 Pass / 0 Fail
Code Coverage	Branch·Default·Reset 방어 분기 추가 보완 필요
Board Validation	UART, SPI, I2C LCD, LED, FND 동작 확인

Functional Coverage와 SPI Bus Coverage는 정의한 목표 항목을 모두 달성하였으며 Scoreboard의 32개 Transaction이 전부 일치하였다. Code Coverage 미도달 영역은 Error, Default, Reset 분기 및 실행되지 않은 방어 로직을 중심으로 추가 Test가 필요하다.

5.5 Board Validation Overview

실보드 검증에서는 Master Board의 MicroBlaze C Application이 Switch와 Button 입력을 읽고 AXI Register를 제어하도록 구성하였다. Slave Board는 UART 또는 SPI로 수신한 데이터를 LED와 FND에 표시하였고, Master Board의 I2C LCD에는 선택된 Mode와 Data를 표시하였다.

- GPIO: Switch·Button 입력과 LED·FND 출력 확인 — PASS
- Custom UART: Master 송신과 Slave 수신 Data 일치 — PASS
- SPI: Master 전송과 Slave 수신 Data 일치 — PASS
- I2C LCD: Mode와 Data 표시 결과 일치 — PASS
- Timer: Counter와 Status Polling 동작 확인 — PASS

5.6 UART & SPI Board-to-Board Validation

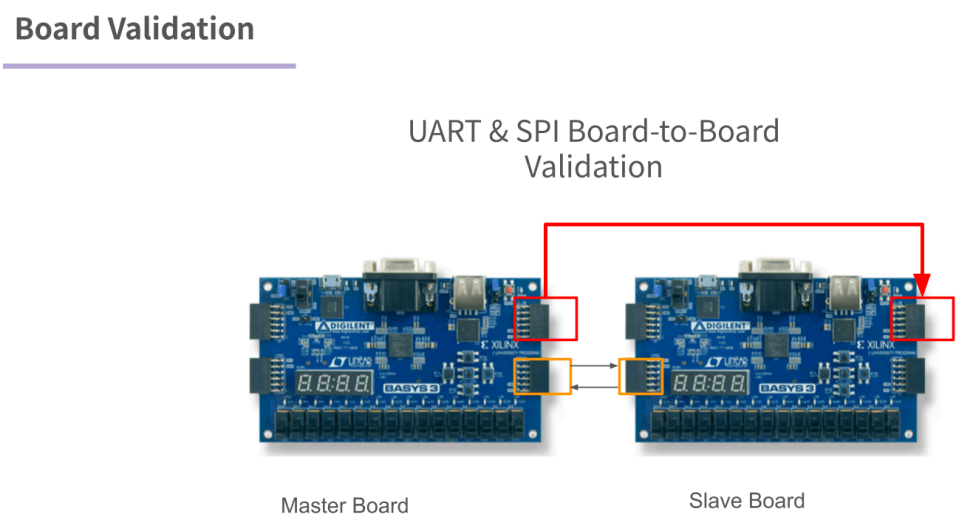


그림 14. Basys3 두 대를 이용한 UART/SPI 연결

- Master Board의 SW[7:0]로 전송 데이터를 설정한다.
- Button 입력으로 UART 또는 SPI Mode를 선택하고 전송을 시작한다.
- MicroBlaze Application이 해당 Peripheral의 Data와 Control Register를 기록한다.
- Slave Board가 Serial Data를 수신하여 LED와 FND에 표시한다.
- Master 입력값과 Slave 표시값을 비교하여 End-to-End Data를 확인한다.

UART는 Master와 Slave의 Baud Rate가 동일해야 정상 수신되었으며, SPI는 CS_n, SCLK, MOSI, MISO의 배선과 Clock Divider 설정을 함께 확인하였다. 이를 통해 Switch 입력에서 Board Output까지 전체 Data Path를 검증하였다.

5.7 I2C LCD Board Validation

Board Validation

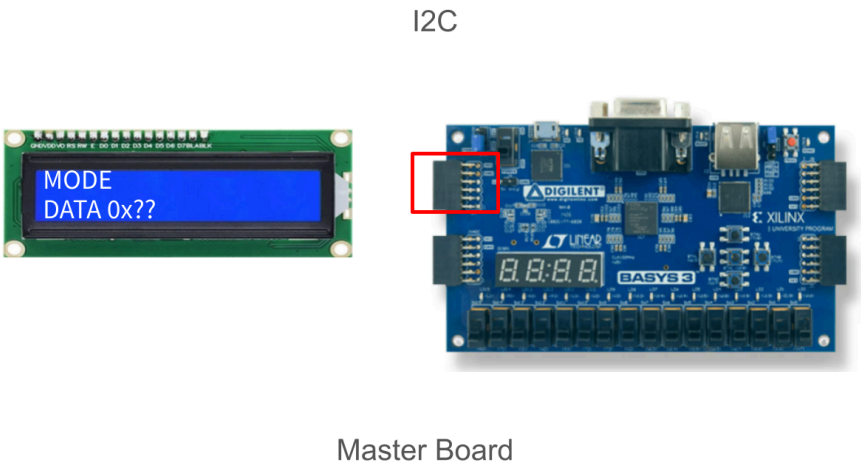


그림 15. I2C LCD Mode/Data 표시 결과

Master Board에 I2C LCD를 연결하여 현재 선택된 Mode와 Data를 표시하였다. Software가 기록한 Display Data가 AXI4-Lite Register, I2C LCD Control Sequence 및 I2C Master를 거쳐 실제 화면까지 전달되는지 확인하였다.

I2C Bus가 Idle 상태에서 SCL과 SDA를 High로 유지하는지 확인한 뒤 START, Address, Command, Data, ACK 및 STOP 순서를 관찰하였다. ACK가 정상이어도 Command 사이 Delay가 부족하면 화면이 갱신되지 않았기 때문에 Protocol Handshake와 LCD Device Timing을 함께 점검하였다.

6. 결과 분석 및 트러블슈팅 (Analysis & Trouble shooting)

6.1 종합 결과 분석

MicroBlaze, AXI4-Lite Interconnect 및 다수의 Custom Peripheral을 하나의 SoC로 통합하고 Vitis Application에서 Memory-Mapped Register 방식으로 제어하였다. SPI Peripheral은 Register-Level과 Protocol-Level을 UVM으로 검증하였고, Board Validation을 통해 Software부터 Physical I/O까지 이어지는 전체 Data Path를 확인하였다.

검증 결과 Functional Coverage 100%, SPI Bus Coverage 100%, Scoreboard 32 Pass/0 Fail을 달성하였다. 또한 UART/SPI Board-to-Board 통신, I2C LCD 표시, GPIO 및 Timer 동작을 실보드에서 확인하였다.

6.2 문제 원인 분석 및 해결

- UART 수신 불일치: Master와 Slave의 Baud Rate 설정이 달라 발생하였다. 양쪽 설정을 동일하게 맞추고 RX_DONE과 RDR 값을 다시 확인하여 해결하였다.
- SPI START 중복 발생: START Bit를 Level로 연결하여 여러 Cycle 동안 동작이 반복되었다. AXI Write Event에서 One-Cycle Start Pulse를 생성하도록 수정하였다.
- DONE 확인 불안정: 완료 상태가 짧게 유지되어 Polling 중 누락될 수 있었다. DONE을 Sticky Status로 유지하고 CLR_STATUS Write로 해제하도록 변경하였다.

- LCD 표시 지연: ACK는 정상이나 LCD Command 사이 Delay가 부족하였다. 초기화 및 Enable Timing을 보완하여 정상 표시를 확인하였다.
- 보드 간 데이터 불일치: 배선과 Reset 순서, CS_n/SCLK 설정을 단계별로 확인하고 입력값 → Register → Serial Bus → 출력값 순서로 비교하였다.

디버깅은 AXI Handshake → Register Value → Core Start/Status → Physical Bus → 최종 Output 순서로 관찰 범위를 좁히는 방식이 가장 효과적이었다.

6.3 개선 및 고도화 방향

현재 Application은 DONE과 STATUS를 반복해서 읽는 Polling 중심으로 동작한다. 구현은 단순하지만 CPU 점유율과 응답 지연 측면에서 비효율적이므로 Timer, UART, SPI 완료 Event를 AXI Interrupt Controller에 연결하고 ISR에서 처리하는 구조로 확장할 필요가 있다.

향후 검증에서는 VALID 유지와 Handshake 안정성, BUSY 중 START 금지, DONE/CLEAR 순서를 SVA로 작성하고 Clock Divider 경계값, Reset Timing, Invalid Address, WSTRB 조합 및 연속 Transaction을 Coverage Cross 항목으로 추가할 계획이다.

또한 Vivado Post-Implementation Report를 기준으로 LUT, FF, BRAM 사용량과 WNS/TNS, 최대 동작 주파수 및 전력을 정량화하여 기능 검증뿐 아니라 Timing과 PPA 관점까지 분석 범위를 확장할 필요가 있다.

7. 결론

7.1 결론 및 고찰

본 프로젝트를 통해 MicroBlaze가 실행하는 C Application과 FPGA 내부 Custom Peripheral이 AXI4-Lite로 연결되는 전체 제어 구조를 설계하고 검증하였다. CPU 관점의 Register Read/Write가 실제 하드웨어에서 Address Decode, VALID/READY Handshake, Control Pulse, Status Update 및 Serial Timing으로 변환되는 과정을 확인하였다.

특히 SPI Peripheral을 대상으로 AXI-Lite Agent와 SPI Monitor를 결합한 UVM 환경을 구성하고 Scoreboard에서 AXI Register Data와 실제 MOSI/MISO Data를 비교하였다. Functional Coverage 100%, SPI Bus Coverage 100%, Scoreboard 32 Pass/0 Fail을 달성하여 Register-Level과 Protocol-Level의 데이터 정합성을 확인하였다.

Basys3 두 대를 이용한 UART/SPI 통신과 I2C LCD 표시를 통해 Simulation을 넘어 실제 배선, Reset, Baud/Clock 설정, Status Polling 및 장치별 Delay까지 포함하는 Board Validation의 중요성을 경험하였다.

7.2 향후 개선 및 고도화 방향

차기 단계에서는 Polling을 Interrupt 기반으로 전환하고 AXI4-Lite Protocol Assertion 및 Corner-Case Coverage를 보강할 예정이다. Register Map을 Header, RTL, UVM에서 동일하게 관리할 수 있도록 자동화하고 Regression 결과를 Seed별로 수집하여 재현성과 분석 효율을 높일 계획이다.

Post-Implementation Timing/PPA 결과까지 포함하여 FPGA Prototype에서 ASIC/SoC 검증 관점으로 분석 범위를 확장하고, Software → AXI4-Lite → Register Block → Peripheral Core → Board I/O로 이어지는 Data Path의 성능과 안정성을 정량적으로 평가할 예정이다.